

Macrolytics

System Architecture & AI Integration

Candidate Profile

Name: Rajeev Lochan Perla

Roll Number: S20230010198

Role: Software Engineer

Submission Overview

This document outlines the problem-solving approach and engineering decisions for the Personal Calorie Tracker assignment. It serves as a technical reference for the decoupled backend architecture, AI agent topologies, AWS data-flow optimizations, and REST API surface.

Contents

1	System Overview	1
2	Core Architectural Optimizations	1
2.1	Decoupled Image Processing & Direct AWS Uploads	2
2.2	Stateless Web Tier via JOSE & JWT Context	2
2.3	Redis Key Isolation & Event-Driven Pub/Sub	2
2.4	SQL vs. NoSQL Database Selection	3
3	Dual AI Agent Architectures	3
3.1	Conversational Chat: Supervisor Multi-Agent Architecture	3
3.2	Background Vision: Evaluator-Optimizer (Critic) Loop	4
4	API Endpoints	4
4.1	Authentication (Public Routes)	4
4.2	Conversational AI & Asynchronous Processing	5
4.3	Nutrition & Health Goals	5
	References	5

1 System Overview

The Macrolytics backend uses a decoupled, event-driven architecture designed to support fast deterministic API requests alongside higher-latency asynchronous AI workloads. Stateless web servers, direct object storage, and queue-backed workers keep request handling separate from storage and compute-intensive inference.

The overall architecture separates the system into four practical concerns: request ingress, stateless application handling, durable/cache data services, and asynchronous AI processing. This separation reduces the coupling between user-facing traffic and long-running inference jobs and provides a clear path to horizontal scaling [1].

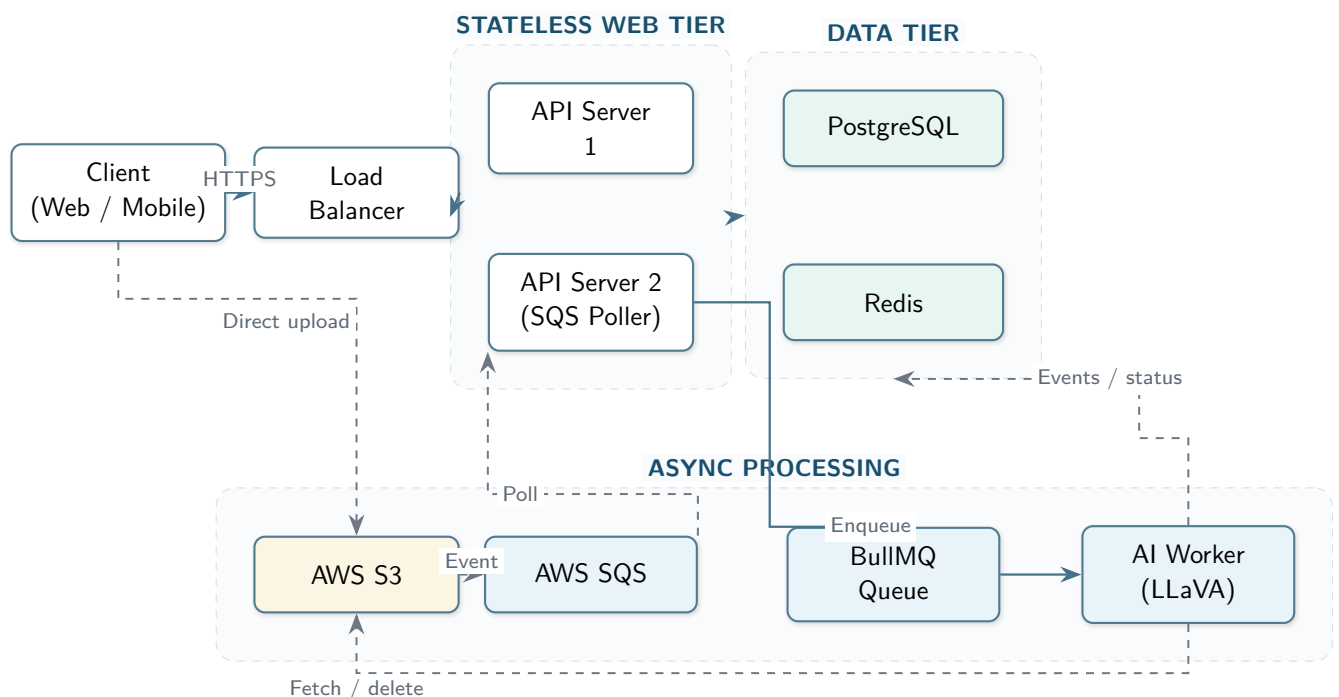


Figure 1: Macrolytics backend architecture. Solid arrows represent the primary request or job path; dashed arrows represent supporting storage and event interactions.

2 Core Architectural Optimizations

The following design decisions address the system's principal scalability constraints: large binary uploads, long-running inference, session state, high-frequency dashboard reads, and transactional persistence.

2.1 Decoupled Image Processing & Direct AWS Uploads

Why this matters

Routing large image binaries through the Node.js event loop increases memory pressure, while local LLaVA inference can take 10–30 seconds per request. The web tier should therefore avoid owning either the binary stream or the GPU-bound work.

- **Implementation:** The client requests a secure pre-signed URL and uploads the binary directly to AWS S3. S3 triggers an `ObjectCreated` event to AWS SQS. A background poller on the API reads this event, creates a “Processing” database stub, and enqueues a lightweight job reference to BullMQ.
- **Justification:** Storage and compute are completely decoupled from the stateless web tier. Heavy file streaming bypasses the API servers entirely, while GPU-bound inference is safely isolated in background workers [1].

2.2 Stateless Web Tier via JOSE & JWT Context

- **Challenge:** Binding conversational state to a specific server introduces sticky-session requirements and weakens horizontal scalability.
- **Implementation:** The system uses a 100% stateless web tier. Chat history and operational context are encrypted with JOSE/JWT using `A256GCM` and returned to the client, which sends the encrypted context on the next request.
- **Justification:** Any API server can handle the next turn without depending on in-memory conversational state [1].

2.3 Redis Key Isolation & Event-Driven Pub/Sub

- **Key design:** Redis stores focused daily macro aggregates rather than attempting to mirror the entire database. The key format is `nutrition:daily:{userId}:{dateStr}`.
- **Atomic operations:** `hIncrByFloat` updates aggregate values atomically, avoiding repeated SQL aggregation work for dashboard reads.
- **Pub/Sub:** Background workers publish completion events to Redis channels. API servers subscribe and can stream status back to the client through Server-Sent Events (SSE) [1].

2.4 SQL vs. NoSQL Database Selection

Selection

PostgreSQL (Relational SQL)

- **Justification:** Nutrition tracking requires transactional integrity, time-range filtering, pagination, and historical aggregation. PostgreSQL provides the relational structure and ACID guarantees needed for consistent macro tracking [1].

Architecture at a glance

Ingress	HTTPS requests enter through the load balancer; large images bypass the API and upload directly to S3.
Web tier	API servers remain stateless and delegate long-running AI work to BullMQ.
Data tier	PostgreSQL is the durable relational store; Redis provides focused aggregates and event fan-out.
AI tier	Background workers fetch images from S3, run LLaVA inference, and validate structured output before persistence.

3 Dual AI Agent Architectures

The system uses two AI topologies, each optimized for a different operational constraint: low-latency deterministic routing for conversation, and high-confidence schema validation for asynchronous image understanding.

3.1 Conversational Chat: Supervisor Multi-Agent Architecture

Standard ReAct-style loops can overload smaller local models when one prompt must simultaneously manage tone, memory, intent classification, and strict JSON formatting. The design therefore uses a **Supervisor Multi-Agent Pattern** [2].

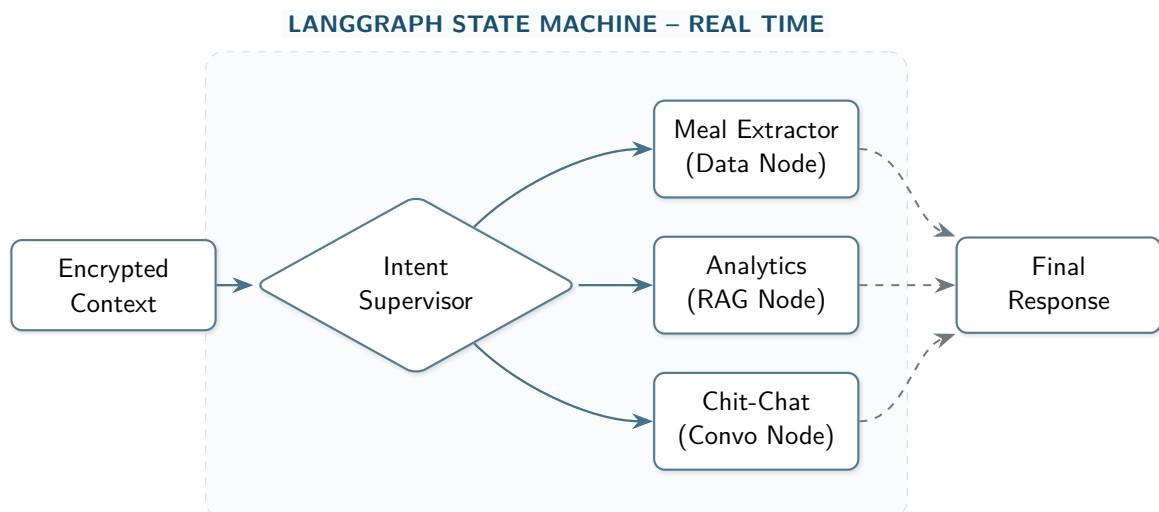


Figure 2: Supervisor routing keeps intent classification separate from specialized sub-agents.

- **Deterministic routing:** The Supervisor uses `withStructuredOutput` for intent classification

and delegates execution to a specialized sub-agent.

- **Cognitive isolation:** Specialized nodes receive constrained prompts without the full conversational overhead, reducing ambiguity during structured operations [2].

3.2 Background Vision: Evaluator-Optimizer (Critic) Loop

Background inference is not limited by interactive response time, so schema correctness becomes the dominant concern. Macro extraction from images therefore uses an **Evaluator-Optimizer loop** [3].

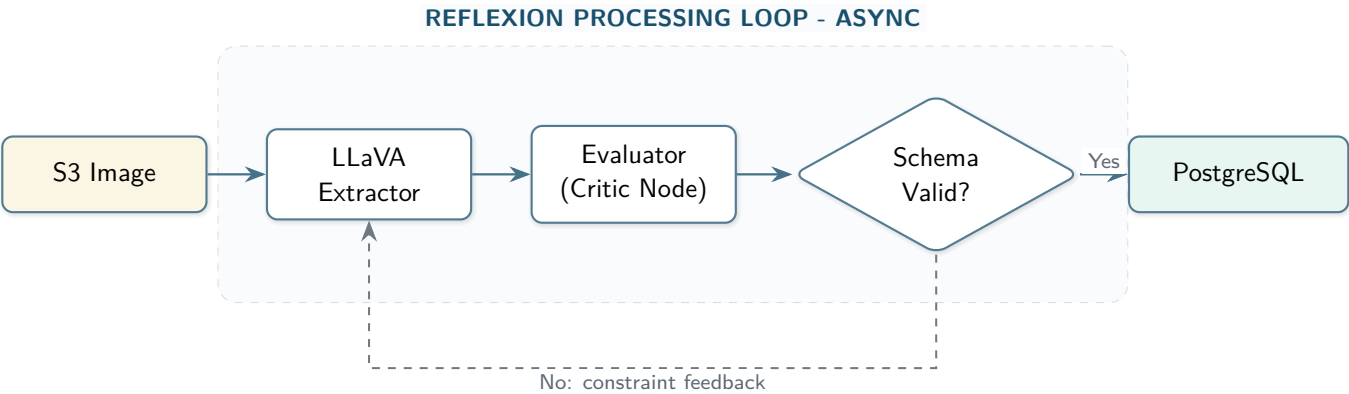


Figure 3: Schema validation creates a controlled retry path for vision-model output.

- **Validation & retry:** A separate programmatic evaluator checks structural integrity. Invalid output is fed back to the vision model for a targeted retry, helping prevent malformed records from reaching the database [3].

4 API Endpoints

The backend exposes a RESTful API with JSON payloads and stateless JWT Bearer Token authorization. The endpoint surface is grouped by responsibility so that the operational role of each route is easy to scan.

4.1 Authentication (Public Routes)

Method	Endpoint	Description
POST	/auth/register	Creates a new user profile and returns a session JWT.
POST	/auth/login	Authenticates a user and returns a session JWT.
POST	/auth/logout	Invalidates the current session token.

4.2 Conversational AI & Asynchronous Processing

Method	Endpoint	Description
POST	/agent/chat	Accepts the <code>encryptedContext</code> token and user prompt. Streams real-time Server-Sent Events (SSE) back to the client.
POST	/uploads/presigned-url	Generates a secure AWS S3 pre-signed URL for direct client-to-cloud uploads, avoiding server memory bloat.
GET	/jobs/:jobId	Polls Redis for the execution status of a BullMQ vision task (Pending, Completed, Failed).

4.3 Nutrition & Health Goals

All list-based endpoints support pagination (`limit`, `page`) and date-range filtering.

Method	Endpoint	Description
POST	/nutrition/entries	Creates a new food entry, executes SQL insert, and increments the daily Redis Hash atomically.
GET	/nutrition/entries	Retrieves paginated food entries. Filterable via <code>startDate</code> , <code>endDate</code> , and <code>mealType</code> .
PUT	/nutrition/entries/:id	Modifies an existing food entry's macros or quantity.
DELETE	/nutrition/entries/:id	Hard deletes an entry and recalculates the daily Redis cache.
GET	/nutrition/goals	Retrieves the user's permanent daily calorie and macro targets.
PUT	/nutrition/goals	Updates the user's customized health goals.
GET	/nutrition/reports/weekly	Returns a 7-day chronological aggregation of Calories, Protein, Carbs, and Fat for frontend charting.

References

- [1] Xu, Alex. *System Design Interview: An Insider's Guide*. ByteByteGo, 2020.
- [2] Wu, Qingyun, et al. "AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation." *arXiv preprint arXiv:2308.08155*, 2023.
- [3] Shinn, Noah, et al. "Reflexion: Language Agents with Verbal Reinforcement Learning." *arXiv preprint arXiv:2303.11366*, 2023.